Functional Programming Practice Variant 2

IT-2512 | Room bookings | 4 tasks | 60 minutes | 40 points

Complete STUDENT CODE and WRITTEN ANSWER Q1-Q4 in the Starter file. Total: 32 points for code and 8 for explanations. Write 2-4 sentences in English per question; code or output alone is insufficient. Answers are graded separately from code.

Timing per task, including the answer: 10 + 10 + 12 + 20 minutes; reserve 8 minutes for reading, checks and saving.

Task 1 Count accepted records

6 points total | Code 4 | Written explanation 2

Count valid records that satisfy either rule: Computer records have Hours less than or equal to maxHours; Lab records have Hours greater than or equal to minHours. Regular records never qualify. Ignore invalid lines and continue processing. Return 0 for empty input or no matches. Assume all numeric parameters are in [2, 10].

Each line has the format Kind;Hours. The provided TryReadLine helper is correct: it accepts Regular, Computer, Lab and an integer from 2 to 10 inclusive, trims both fields, and rejects malformed or null lines. The input sequence itself is non-null.

Correct exactly two logical errors in the method below. Keep the signature and foreach loop.

```
public static int CountAcceptedBookings(IEnumerable<string> lines, int minHours, int maxHours)
{
    int count = 0;
    foreach (string line in lines)
    {
        if (!TryReadLine(line, out string kind, out int value))
            continue;

        if ((kind == "Computer" && value >= minHours) ||
            (kind == "Lab" && value <= maxHours))
            count++;
    }
    return count;
}
```

Example input: "Computer;2"; "Computer;7"; "Computer;3"; "Lab;6"; "Lab;2"; "Regular;8". With minHours = 6, maxHours = 3, expected result: 3.

In DemoTask1, add two Check(label, expectedInt, actualInt) calls. Each input must expose a different bug: fail before the fixes and pass afterwards. Do not reuse the whole provided example.

Required explanation Q1

With minHours = 6 and maxHours = 3, explain why Computer;2 qualifies while Lab;2 does not. Which numeric rule belongs to each category?

CRITERION	POINTS
FIRST LOGICAL ERROR CORRECTED	1
SECOND LOGICAL ERROR CORRECTED	1
TWO DIAGNOSTIC CHECKS EXPOSING DIFFERENT ERRORS	1 + 1
WRITTEN EXPLANATION Q1	2

Task 2 Higher-order scoring

8 points total | Code 6 | Written explanation 2

Provided model

```
public enum RoomKind { Regular, Computer, Lab }
public record Booking(int Id, string Room, int Hours, bool IsConfirmed, RoomKind Kind);
```

Assume valid non-null records: unique positive IDs, non-null text, Hours from 2 to 10, and only declared enum values. All supplied function parameters are non-null. Do not add validation for these assumptions.

Methods to implement

```
public static int Evaluate(Booking item, Func<Booking, int> rule, Func<Booking, bool> condition, Func<int, int> transform)
public static int BaseScore(Booking item)
```

1. Evaluate must calculate rule(item) exactly once. If condition(item) is true, pass the score to transform; otherwise return it unchanged. Do not call transform when the condition is false.

2. BaseScore must use a switch expression on Kind. Select the base points from the table, then add Hours * 2.

KIND	BASE POINTS
Regular	5
Computer	10
Lab	16

3. In DemoTask2, make the two calls shown in the Starter file and print both returned values. Both methods and all supplied functions must be pure: no console or file operations inside them, no input mutation, and no external-state changes.

Sample object: new Booking(99, "Sample", 3, true, RoomKind.Lab). BaseScore(sample) = 22. Expected Evaluate results: 26 and 22.

Required explanation Q2

Suppose rule(item) returns 20 and condition(item) returns false. What should Evaluate return, and should transform be called? Explain what happens when transform is x => x * 3.

CRITERION	POINTS
ONE SCORE AND CONDITIONAL TRANSFORMATION	2
CORRECT BASE MAPPING USING A SWITCH EXPRESSION	2
PURE METHODS AND SUPPLIED FUNCTIONS	1
TWO CORRECT DEMO CALLS AND PRINTED RESULTS	1
WRITTEN EXPLANATION Q2	2

Task 3 Filtered adjacent sums

10 points total | Code 8 | Written explanation 2

For each pair of neighbouring values, calculate their sum. Return that result only if it is less than or equal to maxSum.

public static IEnumerable<int> AdjacentSums(IEnumerable<int> values, int maxSum)

Requirements

Use yield return. Traverse values once per enumeration and keep a fixed number of local variables. Do not use LINQ, indexing, Count(), repeated traversal, a temporary array or list, or shared mutable fields.

The sequence must be lazy: calling the method must not consume input values. Empty input and a single input value must both produce an empty result. Preserve the original order of neighbouring pairs. A skipped pair must not change which values are neighbours in the source sequence.

Assume values is non-null and every integer is from -1000 to 1000. Do not change the input or print inside the iterator. Demonstration calls are already provided; you do not need to write them.

INPUT / PARAMETER	EXPECTED RESULT
4, 1, 6, 2, 8; maxSum=7	5, 7
2, 5, 1; maxSum=5	<empty>
-3, 5, -1, 4; maxSum=3	2, 3
7; maxSum=5	<empty>
<empty>; maxSum=5	<empty>

Required explanation Q3

If the result of a neighbouring pair fails the filter, which source value must become previous for the next pair? Explain why updating previous only when a result is returned gives wrong pairs.

CRITERION	POINTS
CORRECT PAIR RESULTS, FILTER AND ORIGINAL ORDER	3
LAZY GENERATION WITH YIELD RETURN	2
ONE TRAVERSAL AND FIXED EXTRA STORAGE	1
EMPTY INPUT RETURNS AN EMPTY SEQUENCE	1
SINGLE VALUE RETURNS AN EMPTY SEQUENCE	1
WRITTEN EXPLANATION Q3	2

Task 4 Build a ranked report

16 points total | Code 14 | Written explanation 2

Use the provided model and sample data. This task does not depend on your solutions to Tasks 2 or 3. Implement the method using LINQ for filtering, projection, ordering and limiting.

```
public static string[] BuildReport(IEnumerable<Booking> items, Func<Booking, int> rule, int maxHours, int count)
```

1. Keep only records where IsConfirmed is true.

2. Keep only records where Hours is less than or equal to maxHours.

3. Keep only records where Kind is not Regular.

4. Calculate scores with the supplied rule. Retain each computed score for sorting and output; do not score the same eligible record more than once.

5. Sort by score in descending order. For equal scores, sort by Id in ascending order.

6. Return at most count strings in the exact format "Id: Room [score]". For count = 0 or no matches, return an empty array. If fewer records qualify, return all of them.

Assume valid records as in Task 2, a non-null input, a non-null pure rule, maxHours in [2, 10], and count >= 0. No input mutation or console output is allowed inside BuildReport.

ID	ROOM	HOURS	ISCONFIRMED	KIND
9	Lab A	4	true	Lab
4	Lab B	4	true	Lab
2	Room C	4	true	Regular
1	Room D	6	true	Lab
7	Room E	4	false	Lab
6	Room F	3	true	Computer

With maxHours = 4, count = 3 and this independent rule:

```
x => x.Hours * 2 + (x.Kind == RoomKind.Lab ? 3 : 0)
```

Expected strings, in order:

4: Lab B [11]
9: Lab A [11]
6: Room F [6]

Required explanation Q4

Why must the Kind filter be applied before Take(count)? Why does the final projection reuse Score instead of calling rule again?

CRITERION	POINTS
STATUS FILTER	1.5
INCLUSIVE NUMERIC LIMIT	1.5
ADDITIONAL MODEL FILTER	1
SUPPLIED RULE AND REUSE OF SCORES	2 + 1
PRIMARY ORDERING AND TIE ORDERING	2 + 2
COUNT LIMIT; FORMAT/ARRAY; ZERO COUNT; NO MATCHES	1 + 1 + 0.5 + 0.5
WRITTEN EXPLANATION Q4	2